BLACKSHIP

TECHNICAL MANUAL

FreeBSD Jail Orchestration



Version 0.3.0

First edition — November 2026

Blackship Technical Manual

Version 0.3.0, First edition

Copyright © 2026 The Blackship contributors.

This manual and the Blackship source code are distributed under the BSD 3-Clause License. The authoritative license text is in the project repository.

The cover uses original vector artwork. Blackship takes its name and atmosphere from the broad science-fiction idea of ominous “black ships”; this project and manual are not affiliated with or endorsed by Games Workshop, and contain no Warhammer artwork, logos, characters, or setting text.

Manual scope

This edition documents Blackship 0.3.0. The command line remains the authoritative interface: run `blackship --help`, a subcommand’s help, or `blackship mcp`’s generated manual when using a newer release.

How to read this manual

Chapters 1–4 orient a new operator and lead to a running jail. Chapters 5–9 form the day-to-day reference for configuration, images, networking, storage, operations, and commands. Chapters 10–11 cover integration and the security model. Chapter 12 is organized by symptoms for incident response.

Shell sessions use a dollar sign for an unprivileged prompt and a hash sign for root. Do not type the prompt character. Examples assume FreeBSD 15.1 unless the text says otherwise. Replace pool names, interfaces, releases, addresses, and paths with values appropriate to the host.

Destructive commands are called out explicitly. A dry run is recommended before any dependency-wide lifecycle or reclamation operation.

Contents

How to read this manual **iii**

1 Overview **1**

1.1 Who this is for . . . 1

1.2 What it is not . . . 1

1.3 Major capabilities . . . 2

1.4 A typical workflow . . . 2

1.5 Documentation conventions . . . 2

2 Requirements and Installation **3**

2.1 Supported platform . . . 3

2.1.1 Optional host facilities . . . 3

2.2 Installing a release binary . . . 3

2.3 Installing with Cargo . . . 4

2.4 Checking host capabilities . . . 4

2.5 Data and cache directories . . . 4

2.6 Privilege escalation . . . 5

2.7 Shell completion . . . 5

3 Concepts and Architecture **6**

3.1 Control flow . . . 6

3.2 Manifest, service, and project names . . . 6

3.3 Releases, roots, and images . . . 7

3.4 Lifecycle and state machines . . . 7

3.5 Durable stores . . . 7

3.6 Failure model . . . 8

4 Getting Started **9**

4.1 Create a project . . . 9
4.2 Bootstrap the userland . . . 10
4.3 Create the named network . . . 10
4.4 Validate and preview . . . 10
4.5 Start and inspect the jail . . . 10
4.6 Publish a port . . . 11
4.7 Stop and remove resources . . . 11
4.8 Next steps . . . 12

5 Configuration Reference **13**

5.1 Global configuration . . . 13
5.2 Jail definitions . . . 14
5.3 Resource controls . . . 15
5.4 Network configuration . . . 15
5.5 Health checks . . . 16
5.6 Hooks . . . 17
5.7 Two-phase QoS . . . 17
5.8 Draining and cascade policy . . . 17
5.9 Telemetry . . . 18
5.10 Validation discipline . . . 18

6 Releases, Jailfiles, and Images **19**

6.1 Bootstrapping releases . . . 19
6.2 Line-oriented Jailfiles . . . 19
6.3 TOML Jailfiles . . . 20
6.4 Build, inspect, and validate . . . 21
6.5 ZFS layer cache . . . 21
6.6 Commit and reuse . . . 21
6.7 Armada build references . . . 21

7 Networking and Storage **23**

7.1 VNET backends . . . 23
7.2 Netgraph topology . . . 23

7.2.1 Bridge-name collision . . . 24
7.3 Address and interface ownership . . . 24
7.4 Resolver behavior for ephemeral runs . . . 24
7.5 PF port forwarding . . . 25
7.6 ZFS layout and provisioning . . . 25
7.7 Snapshot workflow . . . 25
7.8 Export, import, and migration . . . 25

8 Operations and Observability 27
8.1 Lifecycle selection and ordering . . . 27
8.2 EVA live reconciliation . . . 27
8.3 Interactive and scripted access . . . 27
8.4 Status, health, and statistics . . . 28
8.5 Supervisor . . . 28
8.6 QoS operations . . . 28
8.7 Audit history . . . 29
8.8 Orphan detection and garbage collection . . . 29
8.9 Telemetry . . . 29
8.10 Operational checklist . . . 29

9 Command-Line Reference 30
9.1 Global options . . . 30
9.2 Lifecycle and host commands . . . 30
9.3 Execution and files . . . 31
9.4 Releases and builds . . . 31
9.5 Networking and storage . . . 31
9.6 Health and orchestration . . . 32
9.7 Machine-readable contract . . . 32
9.8 Exit behavior . . . 33

10 Automation and Integration 34
10.1 Armada orchestration . . . 34
10.2 JSON clients . . . 34
10.3 Rust library . . . 34

10.3.1 Container network and mount policy 35
10.4 Cargo features 36
10.5 MCP server 36
10.6 CI pattern 36
10.7 Versioning expectations 37

11 Security and Reliability **38**
11.1 Security boundary 38
11.2 Least privilege 38
11.3 Environment and path hygiene 38
11.4 Network policy 39
11.5 Storage and secrets 39
11.6 Reliability model 39
11.7 Safe change procedure 39
11.8 Backup and recovery 40
11.9 Auditability 40

12 Troubleshooting **41**
12.1 A jail will not start 41
12.2 RCTL is unavailable 41
12.3 Network creation or attachment fails 42
12.4 The jail has no DNS 42
12.5 A netgraph bridge name is already in use 42
12.6 EVA reports restart-required drift 42
12.7 A build unexpectedly reruns steps 43
12.8 Snapshot rollback is refused 43
12.9 Residual resources remain after a crash 43
12.10Automatic escalation fails 43
12.11JSON automation receives an error 43
12.12An ephemeral run cannot ping 43
12.13When to report a bug 44

A Glossary **45**

B Paths and Files **46**

Colophon **47**

CHAPTER 1

Overview

Blackship is a FreeBSD jail orchestrator. It turns a TOML manifest and optional Jailfiles into repeatable jail lifecycle operations: acquire a userland, create or clone a root filesystem, configure isolation and resources, attach networking, run hooks, supervise health, and reclaim every resource when the jail stops.

The project sits between FreeBSD's native tools and a full cluster platform. It does not replace `jail(8)`, ZFS, PF, rctl, cpuset, or rc; it coordinates them and records enough state to recover from partial failure.

1.1 Who this is for

Blackship is designed for:

- FreeBSD administrators who want declarative, dependency-aware jails;
- application teams running several cooperating services on one host;
- CI systems that need fast, disposable FreeBSD environments;
- management interfaces that need stable JSON rather than scraped text;
- developers who want a Rust library for ephemeral jail workloads; and
- operators who value native FreeBSD facilities over a parallel runtime.

It is particularly effective on ZFS hosts. A bootstrapped release can be snapshotted once and cloned for each jail, making creation fast and space-efficient. The non-ZFS path remains supported, but uses full directory copies and cannot offer snapshots, clones, or ZFS send/receive.

1.2 What it is not

Blackship is not a Linux container compatibility layer, a virtual-machine manager, a multi-host scheduler, or a Kubernetes implementation. A jail shares the FreeBSD kernel with its host. Workloads that need a different kernel, hardware emulation, live migration, or hostile multi-tenant isolation should be evaluated against bhyve or another VM boundary.

Blackship also does not invent a package or service manager inside the jail. Jailfiles run ordinary FreeBSD commands, and managed jails normally boot through `/etc/rc` and stop through `/etc/rc.shutdown`.

1.3 Major capabilities

Declarative manifests
Describe jails, dependencies, hooks, VNET, health checks, resource limits, QoS phases, draining, and storage in `blackship.toml`.

Dependency-aware lifecycle
Start prerequisites first and stop dependents first. Select a jail by name, unambiguous prefix, tag, or `ALL`.

Release and image management
Bootstrap FreeBSD releases from distribution sets or pkgbase, build Jailfiles with a ZFS layer cache, and commit configured roots as reusable releases.

Native networking
Create VNET jails using either `if_bridge`/epair or `ng_bridge`/`ng_eiface`, allocate addresses, configure DNS, and publish services through PF anchors.

Operational control
Apply live-safe changes with EVA, supervise health, enforce rctl and cpuset policy, drain connections, inspect audit history, and garbage-collect orphaned resources.

Integration surfaces
Use versioned JSON envelopes, Armada multi-file orchestration, a Rust library, shell completion, or the optional MCP server.

1.4 A typical workflow

The normal path is deliberately short:

1. install Blackship on a supported FreeBSD host;
2. bootstrap a base release;
3. define one or more jails in `blackship.toml`;
4. create a named network when VNET is required;
5. validate and preview the plan;
6. start the target or the entire dependency graph; and
7. observe, update, snapshot, export, or stop it using the same manifest.

One source of truth

The manifest describes desired jail configuration. Runtime stores beneath the data directory record what Blackship actually created. Do not edit those stores by hand; use the CLI, or reconcile manual host changes with `eva`, `cleanup`, or `gc`.

1.5 Documentation conventions

The manual uses *service name* for the short name in the manifest and *full jail name* for the project-prefixed kernel name. The word *release* means a bootstrapped FreeBSD userland or a committed compatible image, not the host kernel release. A *named network* is a persisted Blackship record that can be recreated after reboot; it is not merely a bridge interface that happens to exist.

CHAPTER 2

Requirements and Installation

2.1 Supported platform

Blackship builds and runs on FreeBSD 15.0 or newer. It uses FreeBSD-specific headers, system calls, ioctls, jail parameters, and network facilities; other operating systems are not supported build targets.

The minimum host provides:

- a FreeBSD 15.0+ kernel with jail support;
- the base userland tools, including `jail`, `jls`, `jexec`, `mount`, and `ifconfig`;
- root authority for operations that create or alter host resources; and
- network access to a FreeBSD mirror when bootstrapping a release.

2.1.1 Optional host facilities

Facility	Purpose
ZFS	Clone provisioning, build-layer cache, snapshots, commit, efficient export, and migration.
PF	Port forwarding, NAT integration, and connection draining.
RACCT/RCTL	Memory, CPU, process, file, and custom resource limits; usage sampling.
VNET modules	Per-jail network stacks through epair/`if_bridge` or netgraph.
cpuset	Explicit CPU pinning or least-loaded core allocation.

RCTL requires the kernel accounting facility at boot. Add the following to `/boot/loader.conf` and reboot before relying on resource policy:

```
kern.racct.enable="1"
```

2.2 Installing a release binary

Download the asset appropriate to the host architecture from the project's release page, extract it, and install the executable on the system path:

```
fetch https://github.com/seuros/blackship/releases/latest/download/blackship-freebsd-
    amd64.tar.gz
tar xzf blackship-freebsd-amd64.tar.gz
sudo install -m 0555 blackship /usr/local/bin/blackship
blackship --version
```

The FreeBSD port under `ports/sysutils/blackship` provides the expected packaging metadata and rc integration for a ports-based deployment.

2.3 Installing with Cargo

Building from source requires a Rust toolchain that satisfies the `Cargo.toml rust-version`, a C compiler, the base system headers, the Protocol Buffers compiler, and LLVM/libclang for dependency bindgen. The repository pins Rust 1.99.1 through `mise.toml`. Blackship compiles a small C helper at build time to obtain ioctl and netgraph constants from the host headers instead of hard-coding ABI values.

```
pkg install rust protobuf llvm
cargo install blackship
```

For repository development:

```
git clone https://github.com/seuros/blackship.git
cd blackship
cargo build --release
sudo install -m 0555 target/release/blackship /usr/local/bin/blackship
```

The default build includes the CLI, telemetry support, and MCP server. Library consumers can omit them with `default-features = false`; Chapter 10 covers feature selection.

2.4 Checking host capabilities

The capability report needs neither a manifest nor root:

```
blackship capabilities
blackship --format json capabilities
```

It reports the Blackship version, JSON schema version, target OS and architecture, kernel-derived features, compiled Cargo features, canonical host tools, and installed privilege escalators. Use it as the first handshake for a management application.

2.5 Data and cache directories

Defaults follow the XDG base directory conventions:

Purpose	Default	Override
Persistent data	`~/.local/share/blackship`	`XDG_DATA_HOME`
Bootstrapped releases	`<data_dir>/releases`	manifest
Download/build cache	`~/.cache/blackship`	`XDG_CACHE_HOME`
Manifest	`./blackship.toml`	`--config`

A system-wide installation commonly sets `data_dir` to `/var/blackship`. Whatever path is chosen must remain consistent across interactive commands and the supervisor.

2.6 Privilege escalation

Library calls never elevate themselves. The CLI can re-execute privileged commands through, in order, FreeBSD `mdo`, `doas`, or `sudo`. The helper path is canonical rather than resolved from an untrusted `PATH`.

```
# Refuse automatic elevation.
blackship --no-escalate up --all

# Elevation may occur, but doas/sudo must not prompt.
blackship --non-interactive up --all
```

`mdo` authorization is supplied by `mac_do` policy and is already non-interactive. The CLI resets root identity and XDG environment variables when it re-executes, preventing privileged state from being written under the invoking user's home by accident.

Bootstrap still needs root

Even when the destination directory is user-writable, extracting a FreeBSD base archive needs root to preserve ownership and special permission bits. Do not work around this by stripping archive metadata; the resulting userland is not a faithful release.

2.7 Shell completion

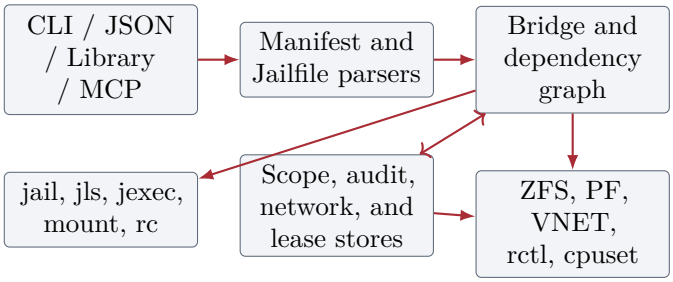
```
blackship completion bash > /usr/local/etc/bash_completion.d/blackship
blackship completion zsh > /usr/local/share/zsh/site-functions/_blackship
blackship completion fish > ~/.config/fish/completions/blackship.fish
```

CHAPTER 3

Concepts and Architecture

Blackship is best understood as a transaction coordinator around native FreeBSD subsystems. It calculates a plan, records ownership, applies host mutations in order, and rolls them back in reverse order if a later step fails.

3.1 Control flow



The CLI is intentionally thin. The same core abstractions back ephemeral library containers and long-lived manifest jails. Errors retain typed causes at the library boundary and stable codes at the JSON boundary.

3.2 Manifest, service, and project names

The manifest is a desired-state document. Each `[[jails]]` table has a short service name. Blackship prefixes that name with the project name when it creates the kernel jail, avoiding collisions between projects. Commands accept the short name, the full name, an unambiguous prefix, a matching tag, or `ALL` where a selector is allowed.

Dependencies form a directed acyclic graph. Starting a target walks its prerequisites in topological order. Stopping a target normally walks its dependents in reverse order. A cycle or unknown dependency is a configuration error, not an ordering guess.

3.3 Releases, roots, and images

A *release* is a complete FreeBSD userland that can seed a jail. It may come from official distribution archives, pkgbase, or `blackship commit`. A jail root is a writable instance of that release.

With ZFS, the release dataset owns a `@pristine` snapshot and jail roots are clones. Without ZFS, Blackship copies the release directory. A Jailfile modifies a root in reproducible steps; on ZFS, each completed `RUN` or `COPY` can become a cached layer snapshot.

3.4 Lifecycle and state machines

The lifecycle distinguishes declared intent from partial host reality. A jail moves through creation, startup, running, draining, stopping, cleanup, and failure states. Blackship checkpoints ownership before each external mutation. Consequently, cleanup can reason about a half-created interface or mount even when no running jail exists.

The main lifecycle operations are:

up provision and start selected jails with their dependencies;
down drain and stop selected jails in dependency-safe order;
restart perform the stop and start plans;
eva reconcile live-safe parameters, resource rules, and CPU placement without restarting; and
gc compare durable ownership with live host state and reclaim or report leftovers.

3.5 Durable stores

Files beneath the configured data directory serve different purposes:

Store	Role
Scope	Current ownership and the rollback work-list for a jail.
Audit	Append-only JSONL lifecycle history that survives scope deletion.
Networks	Persistent named-network definitions, recreated after reboot.
Leases	Serialized address ownership for concurrent jail creation.
Port forwards	PF mappings that can be reloaded after PF restart.
Container registry	Process ownership records for ephemeral library/CLI containers, including explicit ownership handoff.

Mutable TOML state is staged in a uniquely named temporary file beside its target and then atomically renamed into place. Keeping the temporary and target files on one filesystem prevents readers from observing a partially written scope, network, lease, forward, or container-ownership record.

3.6 Failure model

Host operations can fail between any two syscalls. Blackship therefore avoids the fiction that a multi-step jail start is atomic. It instead makes rollback explicit and idempotent. A failed start removes resources it created; a later `cleanup` or `gc` can finish the work after interruption.

Resource-limit and VNET attachment failures are fatal. Running a jail after an isolation control failed would violate the manifest more seriously than returning an error.

Inspect before applying

The structured dry-run plans are calculated from the same dependency and drift logic used by real operations. Use `--format json ... --dry-run` in a UI or `--dry-run` at the terminal before broad changes.

CHAPTER 4

Getting Started

This chapter builds a small VNET web jail backed by ZFS. The examples assume a pool named `zroot`, an external interface already configured by the host, and a FreeBSD 15.1 host.

4.1 Create a project

Make a working directory and generate a starter manifest:

```
mkdir -p ~/jails/web-stack
cd ~/jails/web-stack
blackship armada init
```

Replace the generated content with a minimal configuration:

```
[config]
data_dir = "/var/blackship"
zfs_enabled = true
zpool = "zroot"
dataset = "blackship"

[[jails]]
name = "web"
release = "15.1-RELEASE"
hostname = "web.internal"

[jails.network]
vnet = true
networks = ["default"]
ip = "10.0.1.10"

[jails.healthcheck]
enabled = true

[[jails.healthcheck.checks]]
name = "nginx"
command = "service nginx status"
target = "jail"
interval = 30
```

```
timeout = 10
retries = 3
```

4.2 Bootstrap the userland

```
blackship bootstrap 15.1-RELEASE
blackship releases list
```

On FreeBSD 16 and newer, official distribution sets are no longer the normal source and Blackship uses fingerprint-verified pkgbase automatically. `--pkgbase` opts into that path on supported earlier branches.

4.3 Create the named network

The default epair backend uses an `if_bridge`:

```
blackship network create default \
  --subnet 10.0.1.0/24 \
  --gateway 10.0.1.1 \
  --bridge blackship0
```

To use netgraph instead, choose a distinct bridge name:

```
blackship network create default \
  --subnet 10.0.1.0/24 \
  --gateway 10.0.1.1 \
  --backend netgraph \
  --bridge ngbridge0
```

Do not create both examples under the same network name. Chapter 7 explains the backend tradeoffs and host routing.

4.4 Validate and preview

```
blackship check
blackship up web --dry-run
blackship --format json up web --dry-run
```

The plan should identify the full jail name, root path, address, hook counts, current state, and whether a ZFS dataset will be created. Resolve every validation error before continuing.

4.5 Start and inspect the jail

```
blackship up web
blackship ps
blackship exec web -- uname -a
blackship console web
```

Install and enable a service inside the jail:

```
blackship exec web -- pkg install -y nginx
blackship exec web -- sysrc nginx_enable=YES
blackship exec web -- service nginx start
blackship health web
```

For a reproducible service image, put those installation steps in a Jailfile instead of treating the running jail as the build record; see Chapter 6.

4.6 Publish a port

PF anchors isolate Blackship-managed redirects from the rest of the host's PF configuration:

```
blackship expose web --port 8080 --internal 80
blackship ports web
```

The host PF configuration must load Blackship's anchors. A typical `/etc/pf.conf` contains:

```
rdr-anchor "blackship"
anchor "blackship"
```

Run `blackship setup` after a PF restart, or at boot through the provided rc integration, to reapply persisted networks and forwards.

4.7 Stop and remove resources

```
blackship down web --dry-run
blackship down web
```

`down` stops the running jail and releases runtime resources while preserving its root for future starts. Use `rm --volumes` only when the jail and its associated storage should be removed.

Destructive cleanup

`snapshot rollback --force`, `rm --volumes`, forced garbage collection, release deletion, and recursive ZFS destruction can discard data. Take a snapshot or export first, verify the resolved full jail and dataset names, and prefer Blackship commands over manual host deletion.

4.8 Next steps

Add a database service with `depends_on = ["database"]`, move package installation into a Jailfile, configure a health-driven QoS transition, and run `blackship supervise`. Those features are developed in the following chapters.

CHAPTER 5

Configuration Reference

Blackship reads TOML from `blackship.toml` by default. Select another file with the global `--config` option. Armada can merge several manifests in order; later files override earlier values.

5.1 Global configuration

```toml
[config]
project = "orbital"
data_dir = "/var/blackship"
releases_dir = "/var/blackship/releases"
cache_dir = "/var/cache/blackship"
mirror_url = "https://download.freebsd.org/releases"
zfs_enabled = true
zpool = "zroot"
dataset = "blackship"
bootstrap_archives = ["base", "lib32"]
usage_sample_secs = 30
monitor_ping_url = "https://monitor.example/ping/token"
```

Key	Typical value	Meaning
`project`	string	Prefix for effective jail names. If omitted, Blackship uses the manifest directory name, then the working-directory name, with a path-derived Black Ship name as the final fallback.
`data_dir`	path	Persistent state, roots, scopes, audit, networks, leases, and forward records.
`releases_dir`	path	Bootstrapped and committed release roots.
`cache_dir`	path	Downloads and reusable build material.
`mirror_url`	URL	Base URL for FreeBSD release acquisition.
`zfs_enabled`	boolean	Enable dataset-backed provisioning and ZFS features.
`zpool`	string	Pool containing the Blackship dataset tree.
`dataset`	string	Dataset path below the pool.

`bootstrap_archives`	array	Release components to acquire; the default is `["base"]`.
`usage_sample_secs`	integer	Supervisor racct and telemetry sample period; zero disables sampling.
`monitor_ping_url`	URL	Optional dead-man ping used by the supervisor.

When ZFS is enabled, the jails dataset is mounted at the configured data directory's `jails` path. This keeps effective root paths consistent between directory and dataset deployments.

Operational defaults are tunable in nested tables. The following example also shows bridge VLAN filtering, which requires FreeBSD 15.0 or newer:

```
[config.rate_limit]
jail_start_capacity = 3.0
health_capacity = 5.0
health_refill_rate = 0.5

[config.health]
interval = 30
timeout = 10
start_period = 60
retries = 3

[config.retry]
base_delay_ms = 1000
max_delay_ms = 30000
multiplier = 2.0
max_attempts = 5
jitter_factor = 0.25

[config.bridge]
name = "vswitch1"
vlan_filtering = true

[config.bridge.trunk]
interface = "igb1"
tagged = [10, 20, 99]
disable_hwfilter = false
```

Rate limits bound concurrent starts and health-check dispatch. The `config.health` schema records the intended global check defaults; in version 0.3.0, checks use their own values or the same built-in defaults shown above. Retry values control exponential backoff for HTTP release operations; jitter is expressed as a fraction from zero through one. The optional trunk block declares the physical interface and VLANs admitted by a filtering bridge.

5.2 Jail definitions

Each service is an array entry:

```
[[jails]]
name = "api"
```

```
release = "15.1-RELEASE"
hostname = "api.internal"
depends_on = ["database", "cache"]
tags = ["web", "production"]
devfs_ruleset = 4
init = true
stop = "service api stop"
ephemeral = false
```

`name` is required and must be safe for a FreeBSD jail name. `release` selects the seed userland. `path` may replace the default root path, and `build` or `jailfile` can point Armada at a build context. `hostname` defaults from the service name.

`depends_on` defines graph edges. `tags` groups several jails under a selector. `init = true` runs the FreeBSD rc lifecycle. A manifest `stop` command overrides the Jailfile's `STOP`.

Additional jail parameters are accepted in `params`, but identity and isolation keys managed by Blackship—including name, path, VNET, securelevel, devfs, address, and related `allow.*` controls—cannot be overridden through an escape hatch. Use first-class configuration where provided.

5.3 Resource controls

```
[jails.resources]
memory = "2g"
vmemory = "4g"
cpu = "200"
maxproc = 512
maxthreads = 1024
openfiles = 4096
swap = "1g"
cores = 2
# cpuset = "0,1"
# rules = ["readbps:deny=50m/s"]
```

Named fields become rctl rules. `cpu` is percentage-style pcpu policy; values above 100 permit more than one core of aggregate CPU. `cores` asks Blackship to choose the least-loaded CPUs while `cpuset` pins an explicit list and wins when both are present. The `rules` array accepts advanced raw RCTL resource/action expressions; a jail subject is added when omitted, and a rule targeting a different jail is rejected.

RCTL amount fields accept integers or binary size suffixes `k`, `m`, `g`, `t`, `p`, and `e`, optionally followed by `b`. Blackship expands these to the integer form required by the kernel before issuing the rule syscall.

If configured resource enforcement cannot be applied, startup fails and rolls back. This avoids silently running a workload beyond its declared boundary.

5.4 Network configuration

```
[jails.network]
```

```
vnet = true
networks = ["frontend"]
ip = "10.20.0.15"
# ip_cidr = "10.20.0.15/24" # alternative with explicit prefix
gateway = "10.20.0.1"
mac_address = "58:9c:fc:00:20:15"
vlan_id = 20
backend = "epair"

[jails.network.dns]
mode = "custom"
nameservers = ["10.20.0.53", "1.1.1.1"]
search = ["internal.example"]
domain = "internal.example"
```

A named network supplies subnet, bridge, gateway, and backend runtime data. Explicit values on the jail refine that attachment. Use `ip_cidr` when the prefix must travel with the address, or omit both address keys to use the network lease allocator. Omit `mac_address` for a deterministic address derived from the jail name and bridge. The backend may be `epair` or `netgraph`; VLAN PVIDs are supported by the epair backend on a filtering bridge.

Set `shared = true` for host-network inheritance instead of VNET. This is a weaker isolation mode and is ignored when VNET is enabled.

DNS mode `inherit` copies the host resolver configuration. Mode `custom` writes the declared nameservers, optional search list, and domain into the jail.

5.5 Health checks

All checks are command-oriented: exit zero passes and non-zero fails. The target determines whether the command runs on the host or inside the jail.

```
[jails.healthcheck]
enabled = true

[[jails.healthcheck.checks]]
name = "http"
command = "fetch -qo /dev/null http://127.0.0.1:8080/health"
target = "jail"
start_period = 60
interval = 30
timeout = 10
retries = 3

[[jails.healthcheck.checks]]
name = "listener"
command = "nc -z 127.0.0.1 8080"
target = "jail"
```

The supervisor retains failure history, applies configured recovery behavior, and can use health as the trigger for a QoS phase transition.

5.6 Hooks

```
[[jails.hooks]]
phase = "post_start"
target = "jail"
command = "service nginx reload"
on_failure = "abort"

[[jails.hooks]]
phase = "pre_stop"
target = "jail"
command = "service nginx stop"
on_failure = "continue"
```

Supported lifecycle phases include `pre_create`, `post_create`, `pre_start`, `post_start`, `pre_stop`, `post_stop`, `pre_eva`, and `post_eva`. A phase that requires a running jail is rejected when no JID is available. Use `continue` only when failure is genuinely safe to ignore.

5.7 Two-phase QoS

```
[jails.qos]
transition = "healthy"
max_startup_secs = 120

[jails.qos.startup]
memory = "4g"
cpu = "400"

[jails.qos.steady]
memory = "2g"
cpu = "200"
```

`transition` may be `healthy`, `timer`, or `manual`. Health-driven transition requires the supervisor; `max_startup_secs` provides a deadline fallback. The current phase is recorded so EVA reconciles against it rather than reverting to startup policy.

5.8 Draining and cascade policy

A drain policy blocks new TCP connections through a jail-specific PF sub-anchor, then waits up to its timeout for established states to close before teardown. This is useful for service upgrades behind a load balancer.

```
[jails.drain]
enabled = true
timeout_secs = 30
```

Set `cascade = "strict"`, `"detach"`, or `"isolate"` on the jail entry to control dependents during

`down`:

strict stop dependents first; this is the default;
detach stop only the selected target; and
isolate leave dependents running but fence them from the stopped target.

5.9 Telemetry

```
[telemetry]
endpoint = "http://127.0.0.1:4318"
service_name = "blackship"
metrics_interval_secs = 30

[telemetry.headers]
authorization = "Bearer replace-me"
```

When compiled with telemetry support, the block enables OTLP/HTTP lifecycle events and resource gauges. An unavailable collector warns but does not fail a jail operation. Do not commit credentials in a shared manifest; merge a protected local override when necessary.

5.10 Validation discipline

Always run both syntax/semantic validation and a lifecycle preview:

```
blackship check
blackship armada config --show
blackship up --all --dry-run
```

The first catches manifest and dependency errors, the second reveals merged Armada configuration, and the third resolves actual operational ordering.

CHAPTER 6

Releases, Jailfiles, and Images

Blackship separates the base FreeBSD userland from application-specific image construction. Releases are reusable roots; Jailfiles describe deterministic changes; a running or built jail can be committed back into another release.

6.1 Bootstrapping releases

```
blackship bootstrap 15.1-RELEASE
blackship bootstrap 15.1-RELEASE --archives base,lib32
blackship bootstrap 15.1-RELEASE --pkgbase
blackship releases list
blackship releases verify 15.1-RELEASE
```

FreeBSD 16 and newer use pkgbase automatically. For pkgbase, repository metadata is checked through the FreeBSD fingerprint trust chain. With ZFS, a successful bootstrap creates a release dataset and `@pristine` snapshot. Blackship refuses to treat an incomplete root as a valid release.

6.2 Line-oriented Jailfiles

The default format deliberately resembles a Dockerfile while retaining FreeBSD semantics:

```
FROM 15.1-RELEASE
LABEL service=web version=1.0
ARG NGINX_VERSION=1.26
ENV NGINX_VERSION=${NGINX_VERSION}
RUN pkg install -y nginx
RUN sysrc nginx_enable=YES
COPY nginx.conf /usr/local/etc/nginx/nginx.conf
COPY public/ /usr/local/www/nginx/
WORKDIR /usr/local/www/nginx
EXPOSE 80/tcp
CMD /usr/local/sbin/nginx -g 'daemon off;'
STOP /usr/local/sbin/nginx -s quit
```

Instruction	Meaning
`FROM`	Required base release or committed image.
`LABEL`	Image metadata.
`ARG`	Build-time variable with an optional default.
`ENV`	Environment metadata available to later instructions.
`RUN`	Command executed while building the jail root.
`COPY`	Copy from the bounded build context into the jail root.
`WORKDIR`	Set/create the working directory for later steps.
`EXPOSE`	Document an intended service port; it does not create PF rules.
`CMD`	Default image command metadata.
`STOP`	Command run before rc shutdown; manifest `stop` wins.

Source resolution refuses absolute paths and parent traversal. Destination writes refuse symlink escapes from the jail root. Ownership is resolved against the jail, not the host.

6.3 TOML Jailfiles

The alternative format is convenient for generated configuration:

```
[metadata]
name = "web"
version = "1.0"

from = "15.1-RELEASE"

[[args]]
name = "NGINX_VERSION"
default = "1.26"

[[instructions]]
type = "run"
command = "pkg install -y nginx"

[[instructions]]
type = "copy"
src = "nginx.conf"
dest = "/usr/local/etc/nginx/nginx.conf"

[[expose]]
port = 80
protocol = "tcp"

cmd = "/usr/local/sbin/nginx -g 'daemon off;'"

[stop]
cmd = "/usr/local/sbin/nginx -s quit"
```

Generate either form with `blackship init`; pass `--toml` for the structured variant.

6.4 Build, inspect, and validate

```
blackship template validate Jailfile
blackship template inspect Jailfile
blackship build --file Jailfile --name web --dry-run
blackship build --file Jailfile --name web \
  --build-arg NGINX_VERSION=1.26
```

Dry run does not bootstrap a missing release or mutate storage. Validation should be part of CI before a privileged build job.

6.5 ZFS layer cache

Every cacheable `RUN` and `COPY` contributes to a rolling hash of the base release, earlier instructions, build arguments, and copied content. A completed step is snapshotted as `bs-layer-N-hash`. A rebuild finds the longest matching prefix, rolls back to it, and resumes at the first changed step.

Changing an early instruction invalidates it and all later layers. Changing an unrelated file outside a `COPY` source does not. This makes build contexts both faster and more predictable than timestamp-only caching.

6.6 Commit and reuse

```
blackship build --file runner/Jailfile --name runner
blackship commit runner runner-golden
blackship run --name ci-42 --release runner-golden -- ./run-tests.sh
```

Commit stages the known jail definition and Jailfile under `.blackship/`, snapshots the root, clones it into the release tree, and promotes it so the result is independently reusable. Choose immutable, meaningful release names; consumers should not need to know whether the seed came from FreeBSD or a committed image.

6.7 Armada build references

```
[[jails]]
name = "web"
build = "./web"
depends_on = ["database"]

[[jails]]
name = "worker"
jailfile = "./worker/Worker.jailfile"
```

`build` names a directory containing `Jailfile`; `jailfile` names an explicit file. Armada's `up` builds missing roots by default, `--build` forces a rebuild, and `--no-build` fails when a required root does not already exist.

Builds are not runtime configuration

Put packages, files, and service enablement in a Jailfile. Put host attachment, resource policy, addresses, dependencies, and operational hooks in the manifest. Keeping the boundary clear makes images reusable and manifests auditable.

CHAPTER 7

Networking and Storage

7.1 VNET backends

A VNET jail owns its network stack. Blackship can attach that stack through two native FreeBSD dataplanes:

Property	epair backend	netgraph backend
Host switching	`if_bridge`	`ng_bridge` node
Jail interface	epair peer	`ng_eiface`
Gateway location	bridge interface	host-side `ng_eiface`
VLAN filtering	supported on capable kernels	not supported
Default	yes	no

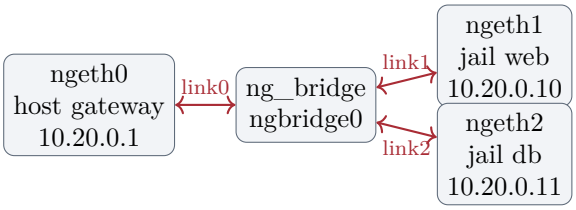
Create a persisted named network rather than manually creating only a bridge:

```
blackship network create frontend \
  --subnet 10.20.0.0/24 \
  --gateway 10.20.0.1 \
  --backend epair \
  --bridge blackship0
```

The record survives reboot; transient interfaces do not. `blackship setup` realizes missing bridges and gateway interfaces from the record.

7.2 Netgraph topology

For a netgraph network, one host-side eiface carries the gateway and each jail receives another eiface on a free `linkN` hook:



Outbound traffic requires host forwarding and usually PF NAT. Blackship enables forwarding while realizing a gateway, but site-specific egress belongs in the host PF policy:

```
nat on $ext_if inet from 10.20.0.0/24 to any -> ($ext_if)
```

7.2.1 Bridge-name collision

Loading `ng_ether` creates netgraph nodes for ethernet interfaces, including `if_bridge` devices. An epair network named `blackship0` can therefore occupy that name in both namespaces. Use a different name such as `ngbridge0` for an actual netgraph bridge.

7.3 Address and interface ownership

Address claims are serialized in the lease store, so concurrent creators cannot receive the same IP. Host-side interfaces are placed in the `blackship` interface group. Cleanup verifies group ownership before destroying an interface found by name, protecting an unrelated interface after name reuse.

The default MAC uses FreeBSD's OUI plus a stable hash of jail and bridge names. It survives restart while differing between clones. Explicitly configure a MAC only when an external network policy requires it.

7.4 Resolver behavior for ephemeral runs

An ephemeral `run` without `--network` inherits the host network stack and copies the host's `/etc/resolv.conf`. A run attached to a named network receives its own VNET stack. Blackship still derives its resolver from the host, but removes loopback nameservers and interface-scoped link-local nameservers that the VNET stack cannot reach.

If filtering leaves no usable nameserver—commonly when the host uses `local_unbound` on `127.0.0.1`—creation fails instead of starting a jail with predictably broken DNS. Supply one or more reachable addresses explicitly:

```
blackship run --name probe --release 15.1-RELEASE \
  --network frontend --dns 10.20.0.53 --dns 1.1.1.1 -- fetch https://example.com
```

The repeated `--dns` values replace, rather than supplement, the host-derived nameserver list. Managed manifest jails continue to use the DNS configuration described in Chapter 5.

7.5 PF port forwarding

```
blackship expose web --port 443 --bind-ip 192.0.2.10
blackship expose web --port 8080 --internal 80 --proto tcp
blackship ports web
blackship unexpose web
```

Blackship writes dedicated PF anchors instead of rewriting the host ruleset. Persisted forward definitions can be reapplied with `setup`. Ensure the main ruleset contains the required `rdr-anchor` and `anchor` statements before publishing a service.

Draining uses a per-jail sub-anchor to reject new TCP SYN packets while allowing existing states to expire. It is not a substitute for application-level readiness removal, but closes the race between a load balancer update and jail teardown.

7.6 ZFS layout and provisioning

With `zpool = "zroot"` and `dataset = "blackship"`, Blackship organizes release and jail datasets beneath that root. Release datasets expose a `@pristine` seed; jails clone from it and mount at the same `data\_dir/jails/name` path used by directory-backed roots.

Advantages include:

- near-instant, copy-on-write provisioning;
- cheap point-in-time snapshots and clones;
- rolling build-layer snapshots;
- zero-copy commit into reusable releases; and
- native send/receive for backup and migration.

7.7 Snapshot workflow

```
blackship snapshot create web before-upgrade
blackship exec web -- pkg upgrade -y

# Rollback requires a stopped jail.
blackship down web
blackship snapshot rollback web before-upgrade --force
blackship up web

blackship clone web@before-upgrade web-test
```

Snapshot and commit operations stage configuration under `.blackship/` inside the dataset so a sent snapshot carries the metadata needed to understand its origin.

7.8 Export, import, and migration

```
# Portable archive.
blackship export web --output web.tar.zst

# ZFS stream.
blackship export web --output web.zfs --zfs-send

# Restore or rename.
blackship import web.tar.zst --name web-restored

# Import a foreign jail layout.
blackship import legacy.tar --from auto --name legacy

# Move a stopped jail over SSH/ZFS receive.
blackship migrate web admin@other-host
```

Foreign import supports iocage, ezjail, plain rootfs, or automatic detection. Review the resulting manifest and networking before first start. Migration requires compatible ZFS and SSH authority on the destination and does not make an incompatible FreeBSD userland/kernel combination safe.

Know the dataset before forcing

Forced rollback destroys newer snapshots. Forced garbage collection or manual `zfs destroy -r` can remove data Blackship cannot reconstruct. List the resolved datasets and take an export before destructive recovery.

CHAPTER 8

Operations and Observability

8.1 Lifecycle selection and ordering

```
blackship up api
blackship up --all
blackship down production
blackship restart api
```

Selectors can be exact service names, full names, unambiguous prefixes, tags, or `ALL`. Starting a target includes its dependencies; stopping uses the configured cascade policy and reverse dependency order.

Preview broad actions before applying them:

```
blackship restart --all --dry-run
blackship --format json restart --all --dry-run
```

8.2 EVA live reconciliation

EVA compares desired parameters with live values from `jls`, applies updatable differences in one `jail_set(JAIL_UPDATE)` operation, replaces managed rctl rules, and re-pins the cpuset. It runs pre/post EVA hooks around the change.

```
blackship eva api --dry-run
blackship eva api
```

Path, VNET mode, addresses, devfs ruleset, and other immutable identity or isolation changes require a restart. EVA reports them but does not pretend to apply them. A jail that is not running is skipped.

8.3 Interactive and scripted access

```
blackship console api --user root
blackship exec api --user www -- id
```

```
blackship exec api --workdir /srv -e MODE=prod -- ./migrate
blackship exec api --clear-env -e HOME=/tmp -- env
blackship cp ./config.toml api:/usr/local/etc/app.toml
blackship cp api:/var/log/app.log ./app.log
```

Use `--` before the in-jail command so its flags are not parsed as Blackship flags. Copy operations reject jail-to-jail transfers and path/symlink escapes.

8.4 Status, health, and statistics

```
blackship ps
blackship stats
blackship stats api
blackship health api
blackship health --watch --interval 5
blackship logs api --lines 200
blackship logs api --follow
```

Statistics aggregate per-process CPU and RSS from `jls` and `ps` libxo JSON. Health status reflects configured checks and retry state. Watch and follow modes are terminal streams; they are intentionally not part of the single-envelope global JSON contract.

8.5 Supervisor

`blackship supervise` runs the warden loop. It monitors jail events, executes health checks, applies recovery/backoff behavior, samples resource usage, performs health-triggered QoS transitions, and can send a dead-man ping. For boot operation, use the packaged rc script with the same manifest and data directory as interactive administration.

The supervisor distinguishes intentional stops from failures and avoids restarting a jail the operator deliberately took down. Each jail has an independent one-for-one restart policy. Abnormal restarts back off from a one-second base toward a 60-second ceiling, with jitter so simultaneous failures do not restart in lockstep. The policy gives up when a jail exhausts five restarts in a five-minute window rather than entering an endless failure loop.

8.6 QoS operations

```
blackship qos api --show
blackship qos api --steady
blackship qos api --startup
```

Only resources named by the target profile are swapped. Custom rules outside the profile survive a phase transition. A one-shot `up` does not remain resident to observe health; use the supervisor or switch manually.

8.7 Audit history

```
blackship audit api
blackship --format json audit api
```

Audit JSONL is durable after the kernel jail, scope record, and even a reboot are gone. The post-mortem summarizes event counts, lifetime, rollback steps, termination reason, and sampled peak resources. Peaks are bounded by the sample interval and are not exact continuous maxima.

8.8 Orphan detection and garbage collection

An out-of-band `jail -r`, crash, or hard reset can leave mounts, interfaces, PF state, leases, rctl rules, or datasets behind. Garbage collection compares durable scopes with live host state:

```
blackship gc --dry-run
blackship gc
blackship gc --force
```

The normal mode reclaims only resources Blackship can prove it owns. Datasets not created by Blackship and non-ephemeral jails are reported rather than destroyed. `--force` expands authority and demands careful review.

8.9 Telemetry

When configured, lifecycle events and racct gauges are exported over OTLP/HTTP. Telemetry runs out of band: collector failure emits a warning but cannot make a jail lifecycle operation fail. This prevents observability from becoming a control-plane dependency.

8.10 Operational checklist

Before a change window:

1. run `blackship check`;
2. inspect `ps`, `health`, and recent `audit`;
3. snapshot stateful ZFS roots or take a portable export;
4. generate the relevant dry-run plan;
5. verify privilege mode and selected targets; and
6. ensure an operator can access the host without depending on the jail being changed.

After the change, verify health, port forwards, dependency status, and audit termination/start records. Run a non-forced GC dry run to catch residual state.

CHAPTER 9

Command-Line Reference

The exact parser generated by the installed binary is authoritative. Use `blackship --help` and `blackship <command> --help` when an option differs from this edition.

9.1 Global options

Option	Effect
`-c, --config <path>`	Select the manifest; default is `blackship.toml`.
`-v, --verbose`	Emit additional operational diagnostics.
`--no-escalate`	Fail privileged commands rather than re-executing through mdo, doas, or sudo.
`--non-interactive`	Add non-prompting behavior to doas/sudo; mdo is inherently policy-based and non-interactive.
`--format text|json`	Select human text or the versioned machine contract for supported commands.

Global options precede the subcommand. Use `--` to terminate option parsing before an in-jail argv.

9.2 Lifecycle and host commands

Command	Purpose
`up [target] [--all] [--dry-run]`	Start selected jails with their dependencies.
`down [target] [--all] [--dry-run]`	Stop in cascade/reverse dependency order.
`restart [target] [--all] [--dry-run]`	Compose stop and start plans.
`eva [target] [--all] [--dry-run]`	Apply live-safe manifest drift.
`ps [--json]`	List configured jail state and JID.
`check`	Validate and resolve configuration.

`capabilities`	Report binary, kernel, feature, facility, and escalator capabilities.
`setup [--enable-pf]`	Recreate networks/anchors and prepare the host; optionally configure and start PF.
`cleanup <jail> [--force]`	Remove resources after a failed lifecycle.
`gc [jail] [--dry-run] [--force] [--prune-unreferenced]`	Find and reclaim orphaned resources; pruning broadens ownership beyond tracked state.

9.3 Execution and files

Command	Purpose
`console <jail> [--user user]`	Open an interactive jail shell.
`exec <jail> [options] -- <argv>`	Execute a command with optional user, working directory, and environment.
`run --name N --release R [--network NET] [--dns IP] [-d] -- <argv>`	Create an ephemeral jail, optionally override its repeatable nameserver list, and clean up unless detached.
`cp <source> <destination> [--preserve]`	Copy host-to-jail or jail-to-host; use `jail:path`.
`rm <jails...> [--force] [--volumes]`	Remove ephemeral/managed jail artifacts and optionally datasets.
`logs <jail> [--follow] [--lines N]`	Read the jail log.

9.4 Releases and builds

Command	Purpose
`bootstrap <release> [--force] [--archives A] [--pkgbase]`	Acquire a base userland.
`releases [list|delete|verify]`	Inspect or manage release roots.
`init [--file F] [--release R] [--toml] [--force]`	Generate a starter Jailfile.
`build [--file F] [--name N] [--build-arg K=V] [--context D] [--dry-run]`	Build an image root.
`template list|inspect|validate`	Work with Jailfiles and templates.
`commit <jail> <release>`	Freeze a jail into a reusable release.

9.5 Networking and storage

Command	Purpose
`network create <name> --subnet CIDR [options]`	Persist and realize a named epair or netgraph network.
`network destroy <name> [--force]`	Remove a managed network.
`network list`	List managed network records and runtime state.
`expose <jail> --port P [options]`	Add a PF port forward.
`ports [jail]`	List persisted forwards.
`unexpose <jail>`	Remove a jail's forwards.
`snapshot create|list|rollback|delete`	Manage jail ZFS snapshots.
`clone <jail>@<snapshot> <name>`	Create a jail root from a snapshot.
`export <jail> [--output F] [--zfs-send]`	Produce a portable archive or ZFS stream.
`import <file> [--name N] [--force] [--from manager]`	Restore native or foreign jail data.
`migrate <jail> <user@host> [--remote-dataset D] [--keep]`	Send a stopped ZFS-backed jail over SSH.

9.6 Health and orchestration

Command	Purpose
`health [jail] [--watch] [--interval N] [--json]`	Run configured checks.
`stats [jail] [--json]`	Aggregate CPU, RSS, and process counts.
`supervise`	Run the warden health/restart/usage loop.
`qos <jail> --show|--startup|--steady`	Inspect or change a QoS phase.
`audit <jail> [--json]`	Render durable lifecycle history.
`armada init|up|down|build|ps|config`	Compose multi-jail workflows and merged manifests.
`completion bash|elvish|fish|nu|zsh`	Generate shell completion.
`mcp`	Serve the optional MCP stdio protocol.

9.7 Machine-readable contract

The global JSON format writes one versioned success object to standard output:

```
{
  "schema_version": 1,
  "ok": true,
```

```
  "data": []
}
```

Failures exit non-zero and write one object to standard error:

```
{
  "schema_version": 1,
  "ok": false,
  "error": {
    "code": "jail_not_found",
    "message": "Jail 'web' not found"
  }
}
```

Stable error codes are for program logic; messages are diagnostic. Commands without a complete contract return `unsupported_output_format` rather than mixing prose into JSON.

Read-only coverage includes status, validation, capabilities, networks, ports, release listing, stats, one-shot health, audit, snapshot listing, and Armada status. Lifecycle JSON is restricted to `--dry-run`. Streaming watch or follow modes cannot fit a single response envelope.

Legacy command-local `--json` flags remain unenveloped for compatibility. New clients should prefer the global format and check `schema_version` before decoding `data`.

9.8 Exit behavior

Zero means the requested operation completed. Non-zero means parsing, validation, escalation, host operation, or rendering failed. An in-jail `exec` propagates the executed process status; treat that result as workload data rather than a Blackship transport error in automation.

CHAPTER 10

Automation and Integration

10.1 Armada orchestration

Armada is the multi-jail, compose-style interface. It can initialize a manifest, merge several files, build missing roots, and apply lifecycle actions to selected services.

```
blackship armada init
blackship armada -f base.toml -f production.toml config --show
blackship armada -f base.toml -f production.toml up --detach
blackship armada ps
```

Later files override earlier values. Keep secrets and host-local paths in a protected final override rather than the shared base manifest. Validate the merged view, not only each input in isolation.

10.2 JSON clients

A management application should begin with:

1. `blackship --format json capabilities`;
2. a schema-version check;
3. `check` against the selected manifest;
4. read-only status calls; and
5. a structured dry run before offering an apply action.

Invoke Blackship with a fixed executable path and explicit working directory. Capture stdout and stderr separately. Decode the envelope only from the stream documented for the exit case. Never parse the human message to determine an error category.

Use `--no-escalate` when the parent application owns authorization, or `--non-interactive` in unattended jobs. An interactive privilege prompt inside a GUI or CI pipe is both confusing and prone to deadlock.

10.3 Rust library

The library API supports ephemeral jails without the CLI, parser, MCP server, or telemetry runtime:

```
[dependencies]
blackship = { version = "0.3", default-features = false }
```

```
use blackship::{ContainerSpec, Exec, Host, Limits, Mount, Network, Selector};
use std::time::Duration;

let host = Host::new("/var/db/ci/blackship")
    .with_zfs("zroot", "ci/blackship");

host.reap_orphans(&Selector::prefix("job-"))?;
host.ensure_release("15.1-RELEASE")?;

let job = host.create(
    &ContainerSpec::new("job-42", "15.1-RELEASE")
        .network(Network::vnet("ci"))
        .nameserver([192, 0, 2, 53])
        .mount(Mount::read_write("/var/db/ci/jobs/42", "/work").noexec())
        .limits(Limits::new().memory("4g").max_processes(512)),
)?;

let result = job.exec(
    Exec::shell("make test")
        .cwd("/work")
        .env("CI", "true")
        .timeout(Duration::from_secs(900)),
)?;

job.destroy()?;
```

`Host` is a clonable description of layout and storage, not a remote connection. Containers own independent host descriptions and can be used from several threads. Dropping a container attempts cleanup; call `destroy` when errors must be observed or `keep` when the instance should outlive the handle permanently. A short-lived privileged creator can instead call `hand_over(pid)`: the receiving process becomes the recorded owner, and orphan reclamation becomes eligible only after that process exits.

10.3.1 Container network and mount policy

`Network::None` leaves the container without a configured attachment. `Network::Inherit` shares the host stack and resolver. `Network::vnet(name)` leases an address on a named network and derives a resolver from the host after dropping host-only loopback and scoped link-local nameservers. Repeated `nameserver(ip)` calls replace the derived list and are required when a VNET container cannot reach the host's local resolver.

Container mounts are nullfs attachments. A read-write `Mount` is `nosuid` by default; `noexec()` adds execution denial, and `allow_setuid()` is an explicit opt-out. These flags protect the jail-side mount only. They do not change how the same files behave through the host path, so writable job directories should live on a host dataset with `setuid=off` when other host users can reach them.

`ContainerSpec::allow_chflags()` is intended for controlled image baking that installs pkgbase, whose packages set system file flags. Do not add it routinely to jobs: flags written through a read-write

mount remain on the host files and can prevent deletion until cleared.

A non-zero executed command is represented in the execution result, while failure to create, enter, supervise, or clean the jail is a typed `blackship::Error`. Library calls require appropriate privilege and never re-execute through an escalation helper.

10.4 Cargo features

Feature	Default	Adds
`cli`	yes	Binary and usage parser; implies telemetry.
`mcp`	yes	MCP host, schemas, tracing, and stdio server; implies CLI.
`telemetry`	through CLI	OTLP export through rama.
`inspect`	no	State-machine introspection.

10.5 MCP server

`blackship mcp` speaks Model Context Protocol over stdin/stdout. It is an authoring and validation surface, not a privileged lifecycle daemon. Its read-only tools expose host facts and the real manifest/Jailfile parsers. File-producing tools render and validate a draft, then require operator elicitation before an atomic write.

The gate fails closed: refusal, timeout, an unsupported client, missing confirmation, a nonexistent destination directory, or failed validation leaves no file behind. Replacing a file requires an additional overwrite decision.

MCP resources include a generated CLI manual, KDL command tree, JSON schemas, Jailfile reference, and reserved jail-parameter reference. Because these are generated from live code, use them when exact current syntax matters.

10.6 CI pattern

A CI adapter can map one job to one ephemeral jail:

1. reap orphaned jobs with a narrow prefix or label selector;
2. ensure the requested release or committed image exists;
3. create a uniquely named jail and bounded mounts/resources;
4. copy or mount the workspace;
5. execute steps with explicit timeouts;
6. copy reports out; and
7. destroy the jail in a finally/defer path.

Timeout and deadline supervision kills the workload from the host side so a process inside the jail cannot evade it. Blackship reaps the timed-out command before sweeping the remaining jail processes, preventing its own zombie from consuming the cleanup retry budget. Use read-only mounts for shared source or caches when the job should not mutate host data.

10.7 Versioning expectations

Pin crate and binary versions in automation. Treat the JSON `schema_version` independently from the Blackship package version. Feature-detect optional behavior with `capabilities`, and never assume a host has ZFS, PF, RCTL, or an authorized escalator solely because another host did.

CHAPTER 11

Security and Reliability

11.1 Security boundary

A FreeBSD jail is operating-system-level isolation, not hardware virtualization. It shares the host kernel. Blackship reduces configuration error and limits resource/network exposure, but cannot strengthen a jail beyond the guarantees of the running FreeBSD kernel and its policy.

Use a VM boundary for untrusted tenants that require independent kernels or for workloads whose compromise impact cannot include the host kernel. Keep the host patched and avoid granting broad jail parameters merely to make an application work.

11.2 Least privilege

Most inspection and authoring operations are unprivileged. Host mutations need root. The CLI's escalation order is mdo, doas, then sudo, using canonical paths. The library never escalates.

For interactive administration, define narrowly scoped `mac_do` or doas policy rather than unrestricted root shells. For a management UI, separate the unprivileged planning process from the small authorized apply path and pass `--no-escalate` to the former.

11.3 Environment and path hygiene

On mdo re-exec, Blackship resets `HOME`, `USER`, and `LOGNAME` to root and removes XDG directory overrides. This prevents a privileged invocation from continuing to use attacker-controlled user paths.

The executable and escalators should be installed in root-owned locations. Automation should use an absolute Blackship path, a known working directory, and an explicit manifest. Do not accept an arbitrary manifest path from an untrusted caller and then authorize its lifecycle operations.

Jailfile copies are bounded to the build context and jail root. Symlink and parent-directory escapes are rejected. MCP writes use a temporary file in the target directory followed by rename, preventing an interrupted write from truncating an existing manifest.

11.4 Network policy

VNET offers a separate network stack; shared networking does not. PF rules still belong to the host security policy. Audit the base ruleset, NAT, Blackship anchors, and external interface exposure together.

Port forwarding defaults should bind only where required. A service intended for localhost should not be published on every host address. DNS and gateway values in a manifest are configuration, not authentication.

Blackship verifies interface ownership before cleanup and serializes address leases, reducing races and accidental deletion. Operators should nevertheless avoid manually renaming or reusing managed interfaces while jails are active.

11.5 Storage and secrets

Jail roots, snapshots, exports, build contexts, staged manifests, and ZFS send streams may contain secrets. Protect them according to the most sensitive jail content. A snapshot is not sanitized because the live file was later deleted.

Keep telemetry tokens and local credentials in protected overrides. Review Jailfiles before building: `RUN` is privileged application installation inside a root filesystem and may fetch or execute external content.

Library-container nullfs mounts deserve two-sided review. Read-write mounts are `nosuid` inside the jail by default and can also be made `noexec`; neither property changes access through the original host path. Put writable, user-reachable host directories on a dataset with `setuid=off`. Treat `allow_setuid()` and `allow_chflags()` as narrow exceptions for trusted build workloads, not general compatibility switches.

11.6 Reliability model

Blackship records ownership before mutation and performs reverse-order rollback. Cleanup is designed to be idempotent, because a crash can interrupt either the forward path or rollback itself. Mutable TOML state is written to a unique same-directory temporary file and atomically renamed over the target. Scope records are current work-lists; audit records are durable history.

The project treats isolation failures as fatal. It does not continue after a requested VNET attach or resource policy fails. This fail-closed behavior may surface host misconfiguration earlier than a hand-written jail script, which is the intended result.

11.7 Safe change procedure

1. identify the exact project, target selector, and resolved full jail names;
2. validate the manifest and merged overrides;
3. inspect capabilities and current status;
4. snapshot or export state that cannot be recreated;

5. generate a dry-run plan;
6. apply with non-interactive escalation only when credentials are already available;
7. verify health and network reachability; and
8. inspect audit and a GC dry run for residual resources.

11.8 Backup and recovery

Test restoration, not only backup creation. A complete recovery plan includes:

- the manifest and Jailfiles in version control;
- portable exports or replicated ZFS snapshots for mutable jail data;
- the pool/dataset naming expected on the destination;
- host PF, loader, rc, and escalation policy outside Blackship's data tree;
- release availability compatible with the destination kernel; and
- a procedure to recreate named networks and forwards with `blackship setup`.

Force means proof has been relaxed

Forced cleanup and garbage collection exist for damaged states where ownership cannot be established normally. They are not routine shortcuts. Capture status, scope/audit files, mounts, interfaces, PF anchors, and datasets before forcing; that evidence is often what explains the incident.

11.9 Auditability

Use stable JSON error codes and audit events for automated decisions. Retain the package version and capability report alongside incident logs. Human CLI text is for diagnosis and may evolve; it is not a durable event schema.

CHAPTER 12

Troubleshooting

Start every investigation with the non-mutating facts:

```
blackship capabilities
blackship check
blackship ps
blackship up --all --dry-run
blackship gc --dry-run
```

Add `--verbose` for command diagnostics. Preserve audit and scope data before manual cleanup.

12.1 A jail will not start

1. run `check` for dependency, name, and path errors;
2. verify the selected release with `releases verify`;
3. inspect the dry-run path, address, hooks, and dataset action;
4. confirm RCTL is enabled if resources are configured;
5. inspect named network state and required kernel modules; and
6. run `cleanup <jail>` only after recording the original error.

A VNET or resource-limit error is fatal by design. Fix the host capability or remove the unsupported requirement; do not expect Blackship to run the jail with weaker isolation.

12.2 RCTL is unavailable

Check the read-only loader tunable:

```
sysctl kern.racct.enable
```

It cannot be enabled at runtime. Add `kern.racct.enable="1"` to `/boot/loader.conf` and reboot. After reboot, validate the actual rules with `rctl -l jail:<full-name>`.

12.3 Network creation or attachment fails

```
blackship network list
ifconfig -g blackship
sysctl net.inet.ip.forwarding
pfctl -sr
pfctl -sn
```

For netgraph, inspect node names and avoid reusing an `if_bridge` name. For epair, confirm the bridge and interface modules are available. Verify that the jail address belongs to the named subnet and is not already leased.

If outbound packets leave the jail but not the host, check forwarding and NAT. If inbound forwarding fails, confirm both the Blackship anchor and the anchor load statements in `/etc/pf.conf`; then run `blackship setup` after any PF reload.

12.4 The jail has no DNS

Inspect `/etc/resolv.conf` inside the jail and the manifest's DNS mode. Custom nameservers must be reachable from the jail's subnet. In inherited mode, ensure the host resolver file existed when the root was prepared. DNS failure with working IP connectivity is not a bridge failure.

For an ephemeral `run` on a named network, Blackship removes host loopback and interface-scoped link-local nameservers because its VNET stack cannot reach them. If the host resolver contains only such entries, creation fails with an instruction to supply a reachable server:

```
blackship run --name probe --release 15.1-RELEASE \
  --network default --dns 192.0.2.53 -- host freebsd.org
```

Use repeated `--dns` options for redundancy. Do not point a VNET jail at `127.0.0.1` unless it runs its own resolver there.

12.5 A netgraph bridge name is already in use

`ng_ether` gives ethernet interfaces netgraph node names. If an `if_bridge` named `blackship0` already exists, create the netgraph network as `ngbridge0` or another distinct name. The two bridge types share a naming namespace even though their switching implementations differ.

12.6 EVA reports restart-required drift

Path, VNET, addresses, and certain jail parameters are immutable for Blackship's live-reconciliation model. EVA reports them without applying them. Schedule:

```
blackship restart <jail> --dry-run
blackship restart <jail>
```

If `jls` cannot report a requested key, EVA may label it unreadable and apply an updatable desired value blind. Review that result on the host's exact FreeBSD version.

12.7 A build unexpectedly reruns steps

The layer key includes the base release, prior instructions, build arguments, and copied content. A change early in the Jailfile correctly invalidates later layers. Confirm that generated files in a `COPY` source are stable and that the same build arguments are supplied. Check available ZFS snapshots; the layer cache is a ZFS feature.

12.8 Snapshot rollback is refused

Stop the jail before rollback. If newer snapshots exist, a non-forced rollback protects them. List snapshots, export anything needed, and only then decide whether `--force` is appropriate.

12.9 Residual resources remain after a crash

Run a GC dry run and compare its report with `mount`, `jls`, `ifconfig -g blackship`, PF anchors, rctl rules, and ZFS datasets. Normal GC reclaims proven ownership. Use targeted `cleanup` for one jail. Force only after confirming that a reported resource does not belong to a different live workload.

12.10 Automatic escalation fails

Run the command once with `--no-escalate` to confirm that privilege is the only issue. Check `capabilities` for installed helper paths. For mdo, verify the applicable `mac_do` rule; installation alone does not grant authorization. For unattended execution, ensure doas/sudo policy permits the command without a password and use `--non-interactive`.

12.11 JSON automation receives an error

Confirm the global option precedes the command and that the command supports the contract. Streaming health/watch and log/follow output is not a single JSON envelope. On failure read stderr, check `ok = false`, branch on the stable error `code`, and log the message for operators.

12.12 An ephemeral run cannot ping

Ephemeral `run` jails are intentionally not granted raw sockets, arbitrary socket families, or chflags. Use a managed manifest jail with an explicitly reviewed policy for workloads that require those facilities. Do not broaden all ephemeral jobs to accommodate one diagnostic command.

12.13 When to report a bug

Include the Blackship version, `capabilities` output, FreeBSD release, minimal manifest with secrets removed, exact command, verbose error, relevant audit records, and whether ZFS/PF/RCTL/VNET are enabled. State whether the issue reproduces after a dry-run-only invocation. Never attach private keys, tokens, full production roots, or unsanitized ZFS streams.

APPENDIX A

Glossary

Armada
Compose-style Blackship interface for merged manifests and multi-jail build/lifecycle operations.

Audit log
Append-only JSONL lifecycle history retained beyond cleanup.

Bridge
Both the internal Blackship coordinator and, in networking context, a FreeBSD packet-switching object. Context distinguishes them.

Cascade policy
Rule deciding whether dependents stop, detach, or are isolated when a dependency stops.

Container
Ephemeral jail owned by a Rust-library or `run` process. It is destroyed with its handle unless kept or handed to another live process.

EVA
Live reconciliation of updatable jail parameters, rctl policy, and CPU placement without a jail restart.

Full jail name
Project-prefixed name registered with the FreeBSD kernel.

Jailfile
Reproducible application image recipe in line or TOML form.

Named network
Persistent Blackship record describing subnet, gateway, bridge, and backend; realized again after reboot.

Release
Complete FreeBSD userland or committed compatible image used to seed jail roots.

Scope record
Durable description of resources currently owned by one lifecycle, used as a rollback and reclamation work-list.

Service name
Short jail name written in the manifest.

VNET
FreeBSD virtualized network stack assigned to a jail.

Warden
Blackship supervisor responsible for health, recovery, usage sampling, and QoS transitions.

ZFS clone
Writable dataset created from a snapshot while sharing unchanged blocks with its origin.

APPENDIX B

Paths and Files

Exact paths follow `data_dir`, `releases_dir`, `cache_dir`, and XDG overrides. The table shows conceptual locations; inspect the active manifest before recovery work.

Path or file	Purpose
`blackship.toml`	Default project manifest.
`Jailfile` / `Jailfile.toml`	Application image recipe.
`<data\_dir>/releases`	Directory-backed release roots or mountpoints.
`<data\_dir>/jails`	Jail roots or ZFS mountpoints.
`<data\_dir>/scope`	Per-jail ownership and rollback records.
`<data\_dir>/audit`	Durable per-jail JSONL histories.
`<data\_dir>/networks`	Named-network records.
`<data\_dir>/mcp.log`	MCP diagnostics when the server is used.
`<cache\_dir>`	Downloads and build cache material.
`<jail-root>/.blackship`	Staged jail definition and Jailfile carried by snapshots/exports.
`/boot/loader.conf`	Boot-only RACCT enablement.
`/etc/pf.conf`	Host PF policy and Blackship anchor load points.
`/etc/rc.conf`	Host service enablement, including packaged warden setup.

Recovery rule

Runtime state files are implementation records, not an alternate configuration API. Back them up for diagnosis, but do not edit them to force desired state. Prefer a validated manifest plus `eva`, `cleanup`, `setup`, or `gc`. If manual host repair is unavoidable, capture the original files and audit trail first.

Colophon

This manual is authored in LaTeX and built from discrete chapter sources in `docs/manual/chapters`. The cover illustration is original vector artwork drawn by the document itself with TikZ; it has no external image dependency.

Blackship is distributed under the BSD 3-Clause License. See the project repository for source code, current release notes, and license text.